Recursion

- Three main components to a recursive function
  - Base Case
    - The most simple version of the problem
    - The point at which we want out function to terminate
  - Recursive call
    - Reduce bigger problems into smaller subproblems
    - Trust the recursive leap of faith
  - Figure out how to rebuild the original problem out of the smaller subproblems

List Functions	Dictionary Functions	Built-In Function
index(): returns the index of the first occurrence of the specified element in the list. Throws ValueError if value is not found. **append()**: adds an item to the end of the list. Does not return any value (i.e. returns None). **extend()**: adds all the elements of an iterable (list, tuple, string etc.) to the end of the list. Does not return any value (i.e. returns None). **insert()**: inserts an element to the list at the specified index. Returns None. **pop()**: removes the item at the given index from the list and returns the removed item If no argument is passed, default value of -1 is passed in (index of last item) If passed value is not in range, IndexError: pop index out of range is thrown **remove()**: removes the first matching element (which is passed as an argument) from the list. Returns None. Throws a ValueError if the element doesn't exist.	**get()**: Takes in a keyname and an optional value argument and returns the value of the item with the specified key. If the key does not exist, get() returns value or None if no value was passed in. **items():** returns an object that contains the key-value pairs of the dictionary, as tuples in a list. **keys()**: returns an object which contains the keys of the dictionary as a list **pop()**: removes the item with the specified key from the dictionary and returns the value of the removed item **update()**: inserts the specified items to the dictionary. The specified items can be a dictionary, or an iterable object with key value pairs. **values()**: returns an object that contains the values of the dictionary, as a list	**sum()**: adds the items of an iterable and returns the sum - Parameters: iterable, start(optional-value is 0 if omitted) **max()**: returns the largest argument or element from iterable - Passing in empty iterator throws ValueError **min()**: returns the smallest argument or element from iterable Passing in empty iterator throws ValueError **any()**: returns True if any element of an iterable is True. If not, it returns False. - Returns False if empty iterable is passed in **all()**: returns True if all elements in the given iterable are true. If not, it returns False. - Returns True if empty iterable is passed in **list()**: returns a list in Python. If no parameters are passed, it returns an empty list. If an iterable is passed as a parameter, it creates a list consisting of the iterable's items. - Strings get broken down into characters - For dictionaries, the keys of the dictionary (in random order) will be the items of the list. -Iterators will have each element placed in the list

WWPD(Left) and Environment Diagram(Right)

```
x = [1, 2]

class A:
    x = 3
    y = 4

    def __init__(self, y):
        self.a = y
        self.x = b
        self.__str__ = lambda: str(y)

    def __str__(self):
        return 'BB'

class B(A):
    x = [5, 6]

    def __init__(self, y):
        self.a = x[1]
        y[1] = 8

b = B(x)
a = A(6)

def dash(x):
    return print(self.x)

elastigirl = Link(7, Link(8))
elastigirl.first = elastigirl.rest
```

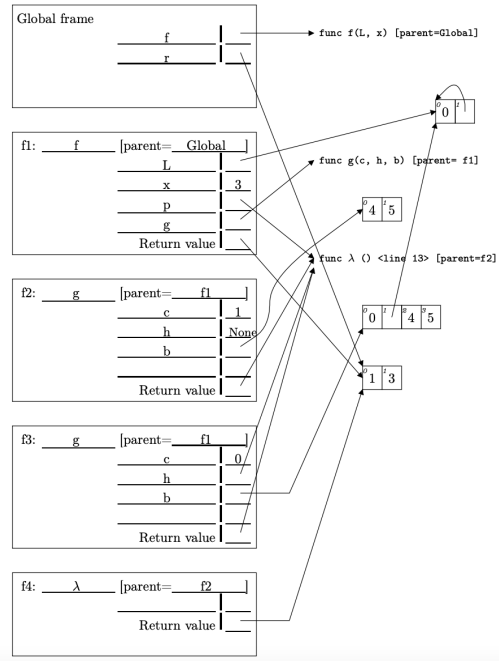
Expression	Interactive Output
pow(10, 2)	100
print(Link(2, Link(3)))	<2, 3>
[c.a for c in [a, b]]	[6, 2]
print(a.x, b.x)	BB [5, 6]
print(b.y, x)	4 [1, 8]
a.__str__()	'6'
dash(b)	Error
print(elastigirl)	<<8>, 8>

2. (3 points) Implement `lowest`, which takes a list of numbers `s` and returns a list of only the elements of `s` with the smallest absolute value. **You may only write a single name in each blank.**

```
def lowest(s):
    """Return a list of the elements in s with the smallest absolute value.

    >>> lowest([3, -2, 2, -3, -4, 2, 3, 4])
    [-2, 2, 2]
    >>> lowest(range(-5, 5))
    [0]
    """
    return (lambda y: [x for x in s if abs(x) == y])(abs(min(s, key=abs)))
```

```
def f(L, x):
  L[1] = L
  def g(c, h, b):
    nonlocal x
    x = c
    if c == 0:
      b.append(5)
      b = L + b
      return h
    else:
      return \
        g(c-1,
          lambda: [c, x],
          b)
  p = g(1, None, [4])
  x += 3
  return p()
r = f([0, 0], 1)
```



Trees and Linked Lists

8. (4 points) Their Mascot is a Tree?!?

Definitions. The *depth* of the root node of a tree is 0, and the depth of a child is 1 more than the depth of its parent. A linked list is either a `Link` instance or `Link.empty`.

Implement `layer`, which takes a `Tree` instance `t` and a non-negative integer `d`. It returns a linked list containing the labels of all nodes in `t` that have depth `d`. Labels in the result should appear in branch order. The `Tree` and `Link` classes appear on the Midterm 2 Study Guide.

You may **not** use `lambda`, `if`, `and`, or `or` in your solution (other than what already appears in the template).

Note: The built-in `reversed` function returns a reverse iterator over the values of a sequence. For example, `list(reversed([1, 2, 3]))` evaluates to `[3, 2, 1]`.

```
def layer(t, d):
    """Return a linked list containing all labels of Tree T at depth D.

    >>> a_tree = Tree(1, [Tree('b', [Tree('mas')]),
    ...                   Tree('a', [Tree('co')]),
    ...                   Tree('d', [Tree('t', [Tree('!')])])])
    >>> print(layer(a_tree, 0))
    <1>
    >>> print(layer(a_tree, 1))
    <b a d>
    >>> print(layer(a_tree, 2))
    <mas co t>
    >>> print(layer(a_tree, 3))
    <!>
    """
    return helper(t, d, Link.empty)


def helper(t, d, s):


    if d == 0:


        return Link(t.label, s)


    else:


        for b in reversed(t.branches):


            s = helper(b, d - 1, s)


        return s
```

6. (4 points) Dr. Frankenlink

Implement `replace`, which takes two non-empty linked lists `s` and `t`, as well as **positive** integers `i` and `j` with `i < j`. It mutates `s` by removing elements with indices from `i` to `j` (removing element i but not removing element `j`) and replacing them with `t`. Afterward, `s` contains all of the objects in `t`, so a change to `t` would be reflected in `s` as well. `t` may change as a result of calling `replace`. Assume `s` has at least `j` elements.

```
def replace(s, t, i, j):
    """Replace the slice of s from i to j with t.

    >>> s, t = Link(3, Link(4, Link(5, Link(6, Link(7))))), Link(0, Link(1, Link(2)))
    >>> replace(s, t, 2, 4)
    >>> print(s)
    <3, 4, 0, 1, 2, 7>
    >>> t.rest.first = 8
    >>> print(s)
    <3, 4, 0, 8, 2, 7>
    """

    assert s is not Link.empty and t is not Link.empty and i > 0 and i < j


    if i > 1:


        replace(s.rest, t, i - 1, j - 1)


    else:


        for k in range(j - i):


            s.rest = s.rest.rest


        end = t


        while end.rest is not Link.empty:


            end = end.rest


        s.rest, end.rest = t, s.rest
```

(a) (6 pt) Implement the `Poll` class and the `tally` function, which takes a choice `c` and returns a list describing the number of votes for `c`. This list contains pairs, each with a name and the number of times `vote` was called on that `choice` at the `Poll` with that name. Pairs can be in any order. Assume all `Poll` instances have distinct names. *Hint*: the dictionary `get(key, default)` method (MT 2 guide, page 1 top-right) returns the value for a `key` if it appears in the dictionary and `default` otherwise.

```
class Poll:
    s = []
    def __init__(self, n):

        self.name = n

        self.votes = {}

        Poll.s.append(self)

    def vote(self, choice):

        self.votes[choice] = self.votes.get(choice, 0) + 1

def tally(c):
    """Tally all votes for a choice c as a list of (poll name, vote count) pairs.

    >>> a, b, c = Poll('A'), Poll('B'), Poll('C')
    >>> c.vote('dog')
    >>> a.vote('dog')
    >>> a.vote('cat')
    >>> b.vote('cat')
    >>> a.vote('dog')
    >>> tally('dog')
    [('A', 2), ('C', 1)]
    >>> tally('cat')
    [('A', 1), ('B', 1)]
    """

    return [(p.name, p.votes[c]) for p in Poll.s if c in p.votes]
```

OOP(Left)

Methods:

Functions within a class that are activated through dot notation.

Examples:

__init__: sets the initialized state of a object instance(involves self (a non-listed parameter)

__repr__: represents the object(when you create an instance, set a variable to an object, etc.) If a class does not have a __str__, then it will return the repr.

__str__: activates when a print is called on an object.

(Methods with two '_' are predefined methods)

(c) (5 pt) Implement `ways`, which takes two values *start* and *end*, a non-negative integer k, and a list of one-argument functions `actions`. It returns the number of ways of choosing functions f_1, f_2, ..., f_j from `actions`, such that $f_1(f_2(...(f_j(start))))$ equals *end* and $j \leq k$. The same action function can be chosen multiple times. If a sequence of actions reaches *end*, then no further actions can be applied (see the first example below).

```
def ways(start, end, k, actions):
    """Return the number of ways of reaching end from start by taking up to k actions.

    >>> ways(-1, 1, 5, [abs, lambda x: x+2])   # abs(-1) or -1+2, but not abs(abs(-1))
    2
    >>> ways(1, 10, 5, [lambda x: x+1, lambda x: x+4])  # 1+1+4+4, 1+4+4+1, or 1+4+1+4
    3
    >>> ways(1, 20, 5, [lambda x: x+1, lambda x: x+4])
    0
    >>> ways([3], [2, 3, 2, 3], 4, [lambda x: [2]+x, lambda x: 2*x, lambda x: x[:-1]])
    3
    """
    if start == end:

        return 1

    elif k == 0:

        return 0

    return sum([ways(f(start), end, k - 1, actions) for f in actions])
```

(a) (6 pt) Implement `partitions`, which is a generator function that takes positive integers `n` and `m`. It yields strings describing all partitions of `n` using parts up to size `m`. Each partition is a sum of non-increasing positive integers less than or equal to `m` that totals `n`. The `partitions` function yields a string for each partition exactly once.

You may **not** use `lambda`, `if`, `and`, `or`, lists, tuples, or dictionaries in your solution (other than what already appears in the template).

```
def partitions(n, m):
    """Yield all partitions of N using parts up to size M.

    >>> list(partitions(1, 1))
    ['1']
    >>> list(partitions(2, 2))
    ['2', '1 + 1']
    >>> list(partitions(4, 2))
    ['2 + 2', '2 + 1 + 1', '1 + 1 + 1 + 1']
    >>> for p in partitions(6, 4):
    ...     print(p)
    4 + 2
    4 + 1 + 1
    3 + 3
    3 + 2 + 1
    3 + 1 + 1 + 1
    2 + 2 + 2
    2 + 2 + 1 + 1
    2 + 1 + 1 + 1 + 1
    1 + 1 + 1 + 1 + 1 + 1
    """

    if n == m:

        yield str(m)

    if n > 0 and m > 0:

        for p in partitions(n - m, m):

            yield str(m) + ' + ' + p

        yield from partitions(n, m - 1)
```

(b) (1 pt) Circle the order of growth of the value of `len(list(partitions(n, 2)))` as a function of `n`. For example, the value of `len(list(partitions(3, 2)))` is 2, because there are 2 partitions of 3 using parts up to size 2: $2+1$ and $1+1+1$. Assume `partitions` is implemented correctly.

Constant Logarithmic Linear Quadratic Exponential None of these

Examples, ways (Left), yield_partitions(right)

Efficiency

Constant: number of operations the function takes is the same

Logarithmic: number of operations the function takes increases with the size of the input, but not at a proportional rate

Linear: the number of operations the function takes is proportional to the size of the input

Quadratic: the number of operations the function takes is proportional to $size_of_input^2$(nested for loops)

Exponential:# operations a function is proportional to $some_number^{size_of_input}$

Category	Theta Notation	Growth Description	Example
Constant	$\Theta(1)$	Growth is independent of the input	`abs`
Logarithmic	$\Theta(\log n)$	Multiplying input increments resources	`fast_exp`
Linear	$\Theta(n)$	Incrementing input increments resources	`exp`
Quadratic	$\Theta(n^2)$	Incrementing input adds n resources	`one_more`
Exponential	$\Theta(b^n)$	Incrementing input multiplies resources	`fib`